

## Questions

As with the previous unit, when composing your answers to the following questions regarding strings, it is only required that you answer in terms of TypeScript. However, it is highly recommended that you also take the time to ensure that you can answer as many of the questions as possible in terms of a programming language with which you are already familiar, and if you wish to incorporate that perspective into your submitted answers feel free to do so (though I will only be interested in the TypeScript-related material when grading).

1. How is string concatenation performed? If there are multiple mechanisms, are there any considerations or caveats when choosing one over another?

**With the +. Another option is something called template literals. Template literals are used to keep code readable without breaking strings while using + requires many quotes and plus signs and is much more visually messy. You can also evaluate expressions in template literals.**

2. Is it possible to have expressions evaluated and the resulting values inserted into string literals? If so, what syntax is used to accomplish this?

**Yes, with template literals. They use backticks ` and are in \${}. An example:**

```
let year = 2026;
```

```
let result = `In 5 years it will be ${year+5}`;
```

3. How do you represent a newline in a string literal? A horizontal tab? A space?

**Newline = \n**

**Tab = \t**

**Space = can be typed into a string**

4. How do you convert all the letters of a string into their uppercase equivalents? What about lowercase?

**With toUpperCase() and toLowerCase().**

5. How do you trim leading and/or trailing whitespace characters from a string value?

**There's a method called trim() and trimStart() gets rid of leading whitespace and trimEnd() gets rid of trailing whitespace.**

6. Is there a built-in method for padding a string value with leading or trailing characters (such as spaces)? If so, how is it accomplished?

**padStart() and padEnd(). They add characters to the beginning or end of a string up to a specified point. For example:**

```
let num = 1
```

```
num.padStart(5, 0); <- generates 000001
```

7. Why do methods of the String object return a (potentially modified) copy of the string's value rather than just modifying the String object itself?

**Strings are immutable. Immutable means they are unable to be changed so TS can't actually change the value of the String object.**

8. What is Unicode? Is it relevant to this language? If so, how?

**It is "universal character encoding" that assigns a different number to each character, symbol, and emoji for every language. TypeScript uses JavaScript's string system which uses Unicode.**

9. How do you find the length of a string?

**The length property. Example:**

```
let word = "Wow";
```

```
console.log(word.length); <- 3 in console
```

10. How do you identify the starting position of the first occurrence of a specified substring in a larger string? What if you want to find the last occurrence?

**indexOf() and lastIndexOf().**

11. How would you extract a specified substring from within a larger string?

**You use the slice() method. In the parenthesis you put the starting index and ending index of the extraction you want.**

12. Define an application programming interface (API).

**An API is a set of rules that let different applications communicate to one another and share data. It gives rules and operations that another program could use without the need to know how the other software works.**

13. What is the purpose of an API key?

**Unique code used to identify and authenticate an application or user trying to make a request for an API. They are used to regulate who can use certain services and to track how much APIs are being used.**

14. Distinguish between database APIs, operating system APIs, remote APIs, and web-based APIs. Which of these is most commonly used?

**Database API = Lets a program to interact with a database.**

**OS API = Lets software use features by an OS.**

**Remote API= Lets programs running on different computers to communicate over a network.**

**Web API = Type of remote API that uses web technologies, like HTTP. It is the most commonly used.**

15. Distinguish between RPC APIs, SOAP APIs, REST (or RESTful) APIs, and GraphQL APIs.

**RPC APIs = Remote Procedure Call API, lets a program that makes a request be performed on another system.**

**SOAP = Simple Object Access Protocol API, very strict and structured. It is a “messaging protocol” used to let different computers exchange info over a network.**

**REST = Representational State Transfer API, set of architectural constraints, it is not a standard.**

**GraphQL = Lets the client specify what data it wants from the API.**

16. Sometimes it is necessary to use one API to access another API. For example, describe in general terms the purpose of the standard Fetch API (which is supported by Deno) and explain how you might use it to interact with a RESTful API.

**Fetch can be used by TS to make HTTP requests and in Deno you can use fetch(). I could use fetch() to send a request to a RESTful API which would then respond back with returned data I could use for my program.**

17. Identify at least three public RESTful web APIs that look like they might be interesting or useful to you. For each, list its name, primary URL (home page, Github page, documentation page, or whatever seems like a logical place from which to start learning to use that API), a brief description of its capabilities, and a summary of how you might envision putting it to use.

**API 1: <https://www.thesportsdb.com/>**

**This is a sports RESTful API. It provides sports information such as teams, leagues, players, events, and results. I could use it to create a sports program that retrieves football teams, schedules, or results. I am a huge sports fan so this could be used very well with a sports related program for which I need sports DATA.**

**API 2: <https://open-meteo.com/>**

**This is a weather API that I could use to get real time weather in user specified locations. It has forecasts and archives centered around the weather.**

**API 3: <https://api.nasa.gov/>**

**This has NASA data such as astronomy images, information about asteroids, and Earth imagery. I could use it to make a program that retrieves information about objects in space.**

18. What is a data interchange (or data exchange) language (or format) and how do they relate to the use of APIs? What are serialization and deserialization of objects and how do they relate to data interchange languages?

**A data interchange language is a standardized way of representing data. It allows different programs to understand and exchange different data. Serialization is converting an object or data into a format that can be stored/transmitted while Deserialization is the opposite process, turning transmitted format back into a workable object. This relates to**

**data interchange languages because data is frequently exchanged and transferred so they are pivotal processes.**

19. What is JSON and why is it relevant to APIs?

**JSON is a data interchange language. It stands for JavaScript Object Notation and it is a text format for data. It is easy for programs to process and generate. It is relevant to APIs because it is commonly used in web APIs to move data.**

20. What is YAML and why is it relevant to APIs?

**YAML is also a data interchange language and it stands for YAML Ain't Markup Language (Originally Yet Another Markup Language). It uses indentation to organize information instead of depending on brackets and other symbols. It is relevant to APIs because it is commonly used for configuration files.**

21. What is XML and why is it relevant to APIs?

**Another example of a data interchange language. It stands for Extensible Markup Language. It is a markup language that is utilized to represent "structured" data. XML is relevant to APIs because it has been widely used to exchange information between systems (especially with SOAP APIs).**

22. Using the formal specification of JSON at <https://www.json.org/json-en.html>, summarize the JSON syntax in your own words.

**Objects, arrays, strings, numbers, true, false, and null can all be found in JSON data. Objects have name and/or value pairs and are enclosed by { }. Arrays contain ordered values and are enclosed by [ ]. Double quote marks are used for strings and object names. Commas are used to divide values, and a colon is used to separate a name from its value. To build more complex systems, objects and arrays can also be nested inside one another.**

23. Deno supports the standard JSON object for working with JSON data. What are the two methods of this object and how are they used?

**JSON.parse() and JSON.stringify(). JSON.stringify() takes a JavaScript/TypeScript value and converts it into JSON text. JSON.parse() does the opposite.**

24. Although Deno does have modules available to facilitate the process, Deno also makes it possible to access command line arguments that were used to run your program without resorting to a module. Describe the mechanism through which you would accomplish this without using a module.

**Deno has the Deno.args property. Deno.args has an array of strings that represent the arguments entered after the script name during program execution.**

25. How can you generate simple textual output for the user to see using the console object? Where does that output appear?

**console.log(). It appears in the terminal the program is running.**

26. What is the Deno Runtime API and what general types of functionality does it provide?

**It is a collection of APIs provided by Deno. It provides functionality involving command-line arguments, the file system, environment information, processes, permissions, networking, and other system-level operations. For example, Deno.args, which was mentioned before, gives a program access to its command-line arguments.**

27. Use the Deno Runtime API documentation to research the alert(), confirm(), and prompt() functions that it contains. For each of these functions, provide a brief description of what it does. What does the optional second argument passed to prompt() accomplish?

**alert() = a message for the user that waits for user acknowledgment.**

**confirm() = Asks yes or no question and returns true or false.**

**prompt() = Asks user to provide a response and that response is returned. The optional second argument of prompt() provides a default value.**

28. What is the Deno Standard Library and what advantage does it hold over the various third-party libraries you might find?

**Collection of modules for programming tasks with tools for things such as formatting. I think a major advantage it has is that it is heavily regulated and audited by Deno with no outside influence. Various third-party libraries are not reviewed or audited.**

29. The Deno Standard Library has been migrated from its historical location at <https://deno.land/std> to the JavaScript Registry (JSR). Describe the process and syntax one would use to access specific Standard Library functions via JSR, what role the deno.json configuration file plays in this process, and why it is better to use JSR than rely on deno.land/std.

**Using the @std namespace and the jsr: prefix, library packages can now be accessed using JSR. These can be recorded by Deno in the deno.json configuration file, which makes it possible to handle these centrally and offer more “straightforward” import names. JSR is better to use than deno.land/std because the Standard Library has officially moved to JSR, and JSR provides modern package management and versioning for Deno projects.**

30. For each of the following Deno modules, provide a brief and general description of the purpose(s) it is intended to serve: io, path, fs, flags, and fmt.

**io = Provides utilities related to input and output, including working with readers, writers, and streams.**

**path = Provides functions for working with file and directory paths.**

**fs = Provides utilities for working with the file system.**

**flags = Designed for parsing command-line arguments and flags.**

**fmt = utilities for formatting information such as text, byte sizes, durations, colors, and overall formatted output.**

31. Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit.

**The clean code philosophy can be applied to this unit by making programs easy to read and understand. For example, variables should have descriptive names instead of unclear and confusing names (like the use of x or y for numbers). Template literals can make strings easier to understand than complicated and messy concatenation. When working with APIs, the code for requesting data, processing the response, and displaying the results can also be separated instead of putting everything together. API keys should also not be directly exposed in public source code. Overall, the clean code philosophy has taught me the importance of keeping programs simple, readable, and easy to maintain to make modification and understanding from myself and peers achievable.**